I Used to Hate Go

What changed my mind as a Java/Kotlin developer

About me!

Kubilay Karpat

- Software Consultant at Xebia
- Backend development
- Java, Kotlin and Go

Add me on Linkedin:
linkedin.com/in/kubilay-karpat/

Copyright © 2025 Xebia. All rights reserved

1 Where I Start

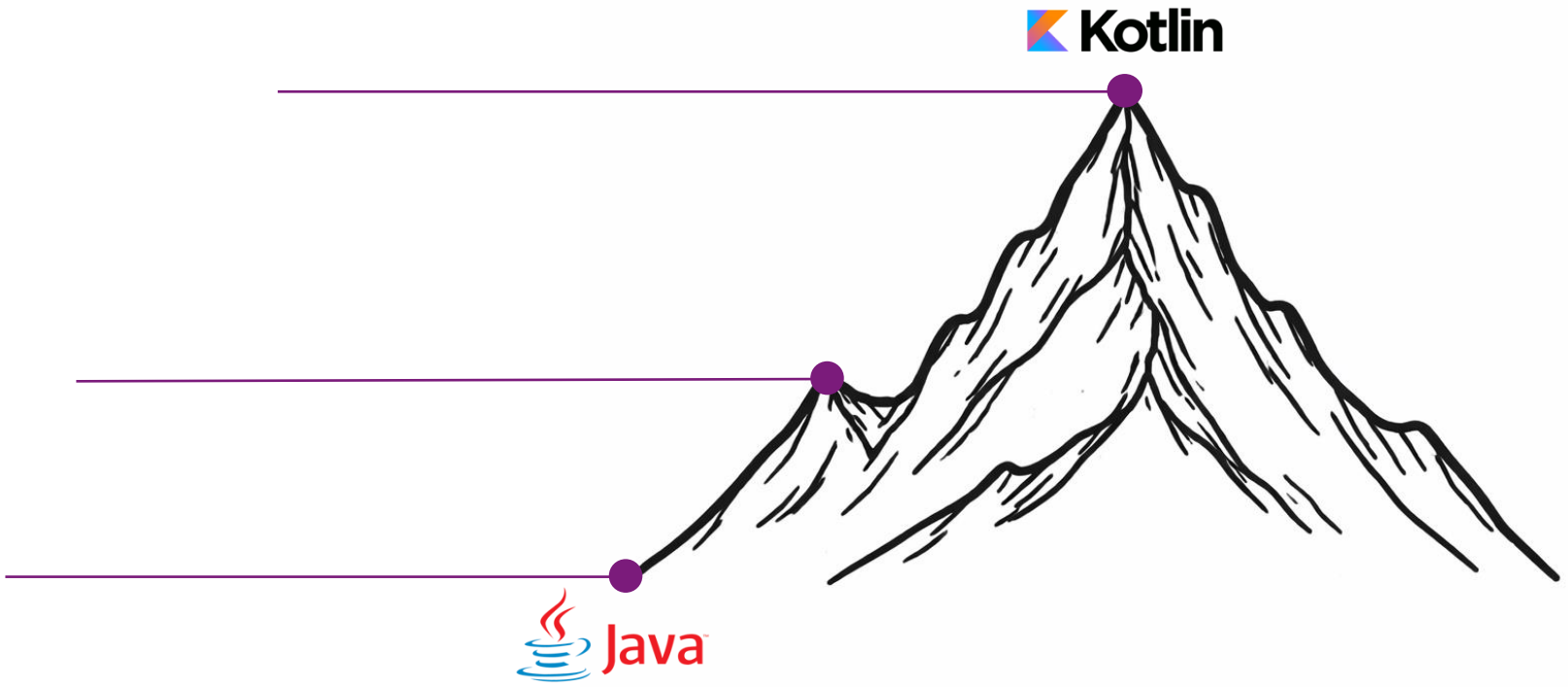
Copyright © 2025 Xebia. All rights reserved

My Journey to the “peak”

2021: Backend Kotlin developer
Cashback/savings apps

2020: Backend Java developer
Payment provider

2016: Full stack Java developer
Jira plugin development



Copyright © 2025 Xebia. All rights reserved

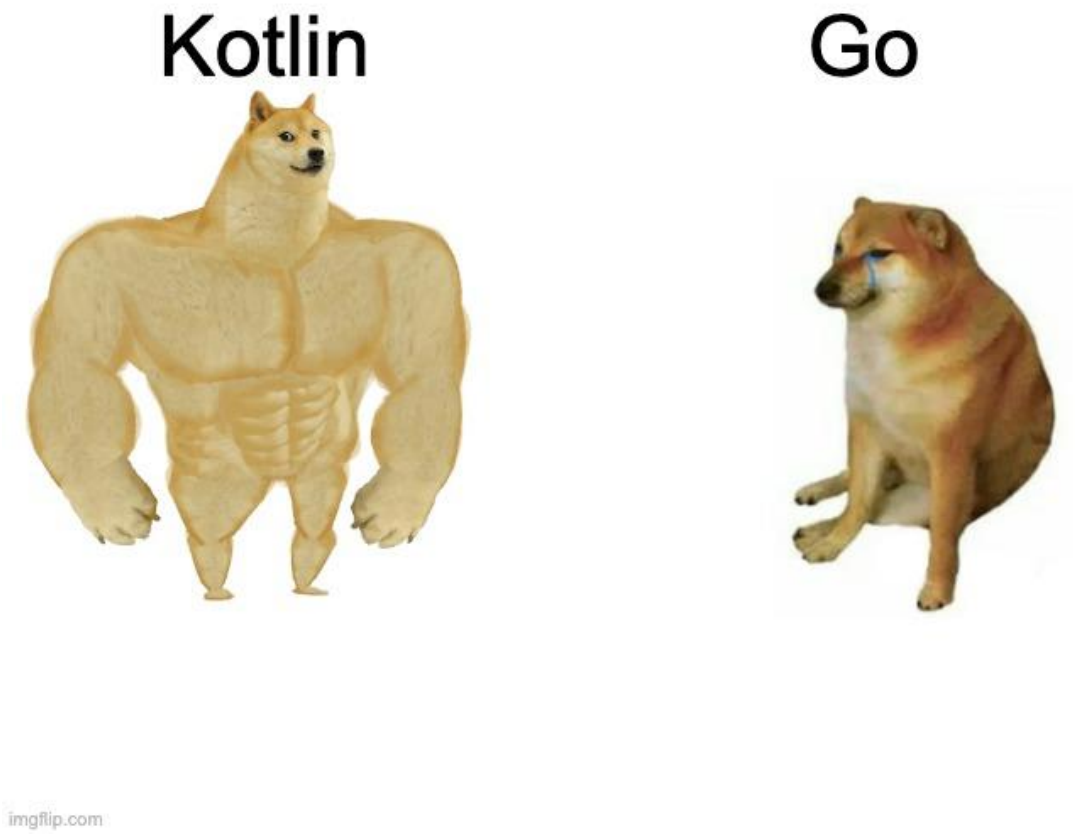
Why I didn’t like Go?

- Verbose
- Fewer abstractions
- Smaller ecosystem

It felt like ”going backwards”

Copyright © 2025 Xebia. All rights reserved

How it felt back then



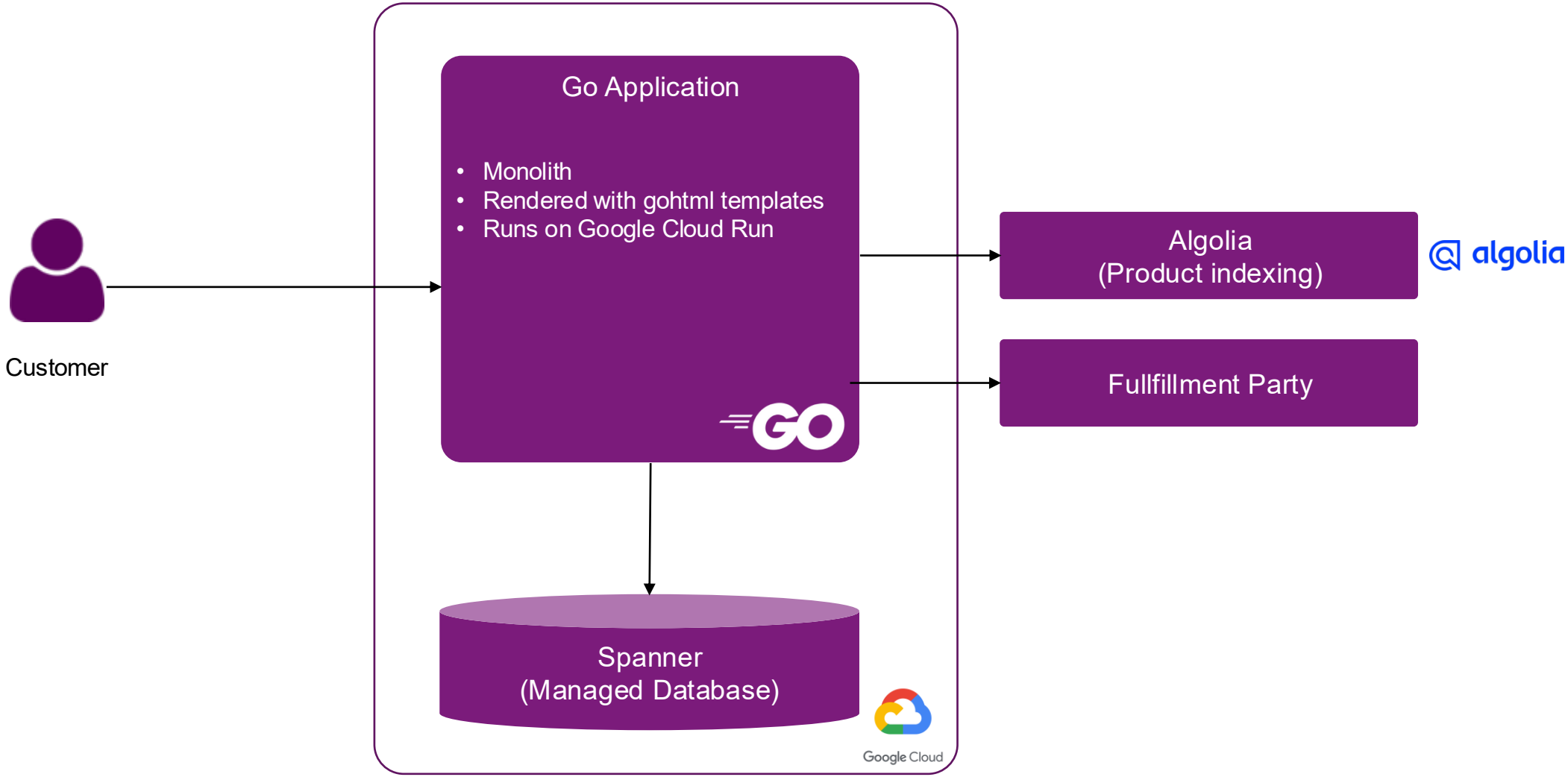
Copyright © 2025 Xebia. All rights reserved

Facing my enemies

- In 2024 I started working at Xebia as a software consultant
- Only available project was a Go project
- I wasn’t sure how long I have to wait for another opportunity

Copyright © 2025 Xebia. All rights reserved

The project that changed everything



Copyright © 2025 Xebia. All rights reserved

What I actually saw:

- visible system shape
- low operational drag
- code that was easy to inspect
- a team comfortable owning the whole product

Go was not the whole reason the team stayed lean,
but it made the lean path easier to choose and harder to accidentally abandon.

Copyright © 2025 Xebia. All rights reserved

2 3 Things Go Does Differently

Copyright © 2025 Xebia. All rights reserved

#1 Go makes it harder to hide complexity

- Application wiring stays visible
- Dependency construction stays visible
- Configuration stays visible
- Startup behaviour stays visible

Framework-driven construction buys convenience. (Java/Kotlin + Spring)

Plain-code construction buys inspectability. (Vanilla go)

On this team, inspectability was worth more.

Copyright © 2025 Xebia. All rights reserved

#1 Go makes it harder to hide complexity

Kotlin + Spring Application Wiring

Application.kt

```kotlin
@SpringBootApplication
class App

fun main(args: Array<String>) {
    runApplication<App>(*args)
}
```

PaymentConfig.kt

```kotlin
@Configuration
class PaymentConfig {
    @Bean
    @Profile("prod")
    fun paymentClientProd(
        @Value("\\${payment.base-url}") baseUrl: String,
        @Value("\\${payment.api-key}") apiKey: String
    ): PaymentClient = RealPaymentClient(baseUrl, apiKey)

    @Bean
    @Profile("dev")
    fun paymentClientStub(): PaymentClient = StubPaymentClient()
}
```

OrderService.kt

```kotlin
@Service
class OrderService(
    private val paymentClient: PaymentClient,
    private val orderRepo: OrderRepository
) {
    fun submit(orderId: String) {
        val order = orderRepo.find(orderId)
        paymentClient.charge(order.total)
    }
}
```

OrderController.kt

```kotlin
@RestController
class OrderController(
    private val orderService: OrderService
) {
    @PostMapping("/orders/{id}/submit")
    fun submit(@PathVariable id: String) {
        orderService.submit(id)
    }
}
```

Copyright © 2025 Xebia. All rights reserved

#1 Go makes it harder to hide complexity

Go Application Wiring

main.go

```go
type Config struct {
    Env, HTTPAddr, PaymentBaseURL, PaymentAPIKey string
}
func mustLoadConfig() Config { /* reads env vars with fallbacks — all explicit */ }

func NewOrderService(payments PaymentClient, repo *OrderRepository, ...) *OrderService

func main() {
    cfg := mustLoadConfig()
    var paymentClient PaymentClient
    if cfg.Env == "prod" {
        paymentClient = NewRealPaymentClient(cfg.PaymentBaseURL, cfg.PaymentAPIKey)
    } else {
        paymentClient = NewStubPaymentClient()
    }
    orderSvc := NewOrderService(paymentClient, NewOrderRepository(), logger)
    r.Post("/orders/{id}/submit", SubmitOrderHandler(orderSvc))
    http.ListenAndServe(cfg.HTTPAddr, r)
}
```

service.go

```go
type PaymentClient interface { Charge(ctx context.Context, amount int64) error }

func NewOrderService(payments PaymentClient, repo *OrderRepository) *OrderService {
  return &OrderService{ payments: payments, repo: repo}
}

func (s *OrderService) Submit(ctx context.Context, orderID string) error {
  order,err  := s.repo.find(orderID)
  if err != nil { return fmt.Errorf("finding order: %w", err) }
  return s.payments.Charge(ctx, orderID)
}
```

routes.go

```go
func SubmitOrderHandler(svc *OrderService) http.HandlerFunc {
  return func(w http.ResponseWriter, r *http.Request) {
    if err := svc.Submit(r.Context(), chi.URLParam(r, "id")); err != nil {
      http.Error(w, "submit failed", http.StatusInternalServerError)
      return
    }
    w.WriteHeader(http.StatusNoContent)
  }
}
```

Copyright © 2025 Xebia. All rights reserved

#2 Explicit error and transaction boundaries force decisions into the open

- Error handling is explicit
- Retries do not happen by magic
- Fallbacks do not happen by magic
- Transaction scope does not happen by magic

Copyright © 2025 Xebia. All rights reserved

#2 Explicit error and transaction boundaries force decisions into the open

Product page can degrade. Checkout cannot.

product.go

```go
price, err := h.prices.LivePrice(r.Context(), id)
if err != nil {
    h.logger.Warn("...", err)

    price, err = h.cache.CachedPrice(r, id)
    if err != nil {
        http.Error(w, "...", http.StatusServiceUnavailable)
        return
    }
}
// Stale price may be acceptable here.
```

checkout.go

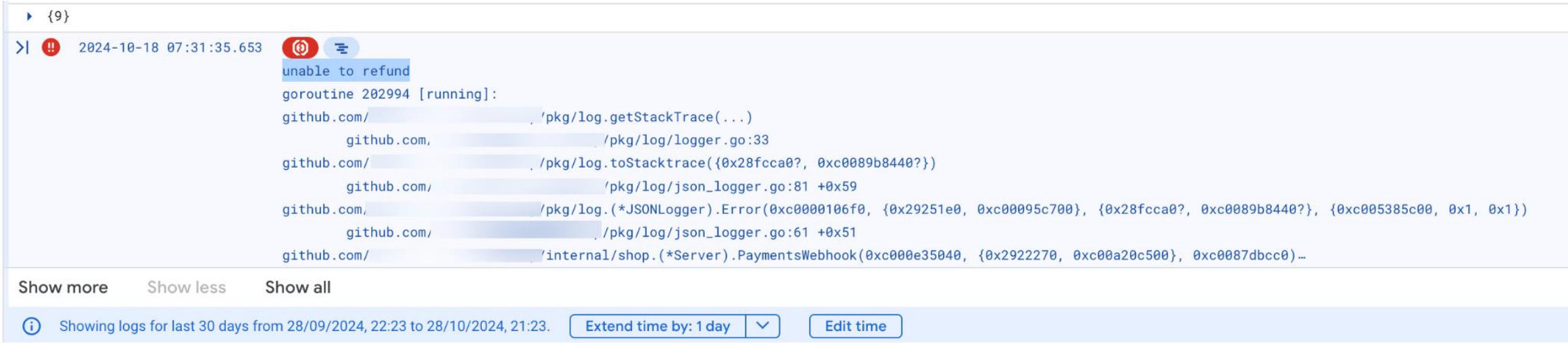
```go
price, err := h.prices.LivePrice(r.Context(), id)
if err != nil {
    h.logger.Error("...", err)

    http.Error(w, "...", http.StatusConflict)
    return
}


// Same failure. No fallback here.
// Policy is visible, local, explicit.
```

Copyright © 2025 Xebia. All rights reserved

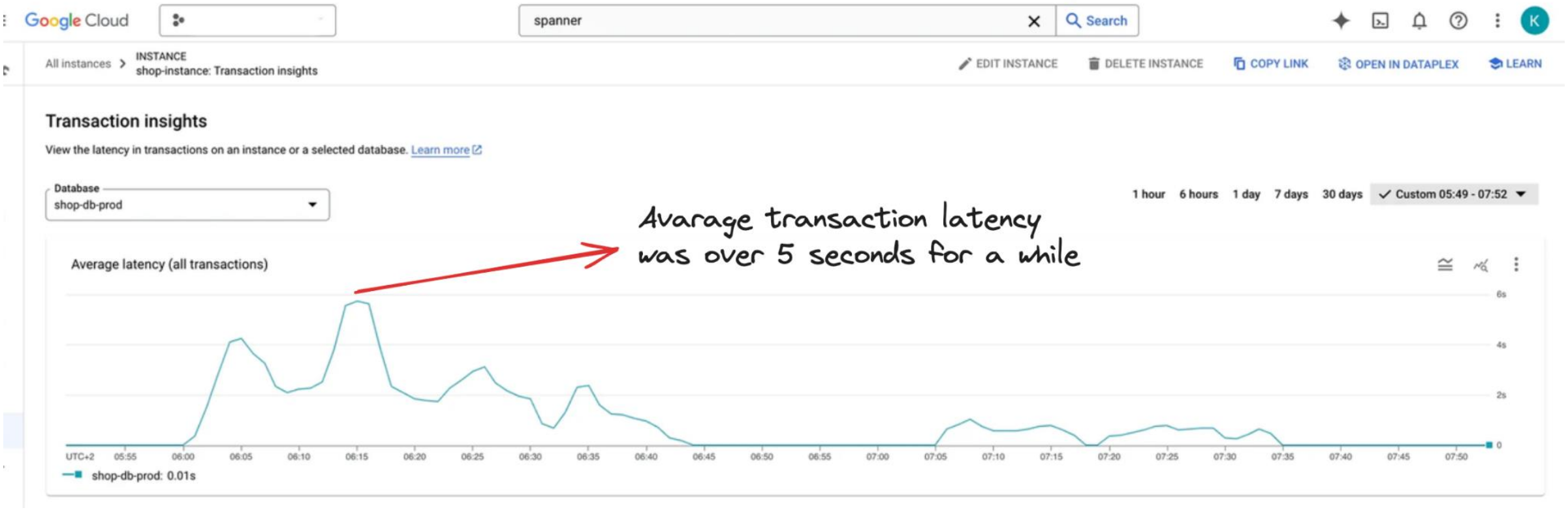
Production incident: a slow query turned retries into duplicate refunds



Some refund payments requests has been rejected by Adyen

Copyright © 2025 Xebia. All rights reserved

Production incident: a transaction bug locked the database



Copyright © 2025 Xebia. All rights reserved

Production incident: a transaction bug locked the database

db.go

```go
func main(){
  ...
  messaging.Subscribe(ctx, "refund_payments", func(ctx context.Context, msg RefundEvent){
    refund, err := db.Query("SELECT * FROM refunds WHERE refund_id = ?", msg.RefundID)
    if err != nil { return err }

    if refund.PaymentReference != "" { return nil }

    paymentRef, err := adyenRefund(ctx, refund)
    if err != nil { return err }

    err = db.ReadWriteTransaction(ctx, func(ctx context.Context, tx *sql.Tx) error {
      _ , err := tx.Exec("UPDATE refunds SET paymentRef = ? WHERE refund_id = ?",
                 paymentRef, msg.RefundID)
      return err
    })

    if err != nil { return err }
    return nil
  })
  ...
}
```

Bug #1 Inefficient query caused table-wide locks
refunds table became slow and blocked writes

Bug #2 Missing idempotency key
retries re-issued the refund
PSP treated each retry as a new request

Result: refund succeeded but was not recorded
refund payments happen but not marked as done

Copyright © 2025 Xebia. All rights reserved

Production incident: a transaction bug locked the database

db.go

```go
func main(){
  ...
  messaging.Subscribe(ctx, "refund_payments", func(ctx context.Context, msg RefundEvent){
    refund, err := db.Query("SELECT * FROM refunds WHERE refund_id = ?", msg.RefundID)
    if err != nil { return err }

    if refund.PaymentReference != "" { return nil }

    paymentRef, err := adyenRefund(ctx, refund)
    if err != nil { return err }

    err = db.ReadWriteTransaction(ctx, func(ctx context.Context, tx *sql.Tx) error {
      _ , err := tx.Exec("UPDATE refunds SET paymentRef = ? WHERE refund_id = ?",
                 paymentRef, msg.RefundID)
      return err
    })

    if err != nil { return err }
    return nil
  })
  ...
}
```

Event handler scope
Retries entire operation on any error

Database read transaction scope

Read-write transaction scope
Retries database work on error

Copyright © 2025 Xebia. All rights reserved

Production incident: a transaction bug locked the database

In Java + Spring

- Transaction scope often starts at @Transactional
- Framework and proxy behavior matter
- Reconstruction cost is higher

In Go

- Transaction scope usually starts where code says it starts
- Control flow is local
- Debugging path is shorter

The bug was ours. What Go improved was the debugging surface.

Copyright © 2025 Xebia. All rights reserved

#3 Dependency culture reduces maintenance drag

- Smaller dependency surface
- Less dependent on any framework
- Boring updates

A lean team does not just need fast feature delivery.

It also needs upgrades and maintenance to be boring.

Copyright © 2025 Xebia. All rights reserved

#3 Dependency culture reduces maintenance drag

In the Go project:

- a small set of major dependencies
- external client libraries for payment/search/fulfillment
- router
- database client
- Most updates uneventful

In JVM Projects

- Major version updates are painful
- Breaking changes are plenty
- Common to see projects stuck in very old version stacks
- Framework versions adds another level complexity

Copyright © 2025 Xebia. All rights reserved

3 What I Still Miss

Copyright © 2025 Xebia. All rights reserved

What I still miss from richer languages

JSON APIs: omitted vs null vs zero value

```go
type UpdateUserRequest struct {
    DisplayName    string `json:"displayName"`
    Phone          string `json:"phone"`
    MarketingOptIn bool   `json:"marketingOptIn"`
}

// Was marketingOptIn omitted, or set to false?
// Was displayName omitted, or intentionally cleared?
```

```go
type UpdateUserRequest struct {
    DisplayName    *string `json:"displayName"`
    Phone          *string `json:"phone"`
    MarketingOptIn *bool   `json:"marketingOptIn"`
}

// Distinguishable — but now everything
// is a pointer. Nil checks everywhere.
```

- Transport structs get noisy
- Validation gets noisier
- Business code gets nil checks throughout

Copyright © 2025 Xebia. All rights reserved

What I still miss from richer languages

Rich domain modeling: where sealed hierarchies help

PaymentResult.kt

```kotlin
sealed interface PaymentResult {
    data class Authorized(
        val authId: String,
        val expiresAt: Instant) : PaymentResult
    data class Declined(val reason: String)
    data class Requires3DS(val redirectUrl: String)
    data class Failed(val errorCode: String)
}
fun handle(result: PaymentResult) = when (result) {
    is Authorized   -> capture(result.authId)
    is Declined     -> showDecline(result.reason)
    is Requires3DS  -> redirect(result.redirectUrl)
    is Failed       -> logFailure(result.errorCode)
}  // Exhaustive — compiler enforced.
```

payment_result.go

```go
type PaymentStatus string
const (
    PaymentAuthorized  PaymentStatus = "authorized"
    PaymentDeclined    PaymentStatus = "declined"
    PaymentRequires3DS PaymentStatus = "requires_3ds"
    PaymentFailed      PaymentStatus = "failed"
)
type PaymentResult struct {
    Status      PaymentStatus
    AuthID      string    // only valid when Authorized
    Reason      string    // only valid when Declined
    RedirectURL string    // only valid when Requires3DS
    ErrorCode   string    // only valid when Failed
}
// Invalid combinations are easy to construct.
```

In Go, the usual answer is disciplined constructors, validation, and tests.
That works. It is just less help from the compiler.

Copyright © 2025 Xebia. All rights reserved

4 What Actually Changed My Mind

Copyright © 2025 Xebia. All rights reserved

What actually changed my mind

Lesson 1: Asking the right questions

I used to ask:

Which language lets me express more?

Now I **also** ask

Which language helps a team safely understand, change and run the whole system?

Copyright © 2025 Xebia. All rights reserved

What actually changed my mind

Lesson 1: Asking the right questions

Lesson 2: There is no best language but tradeoffs

- Best language and tools depends on the team, project and many other factors
- Often there could be more than one good solution for same circumstances

Copyright © 2025 Xebia. All rights reserved

What actually changed my mind

Lesson 1: Asking the right questions

Lesson 2: There is no best language but tradeoffs

Lesson 3: Adapting a new language extends your views on other languages

- Working with Go made me realise design choices and tradeoffs in Kotlin and Java better

Copyright © 2025 Xebia. All rights reserved

What actually changed my mind

I Used to Hate Go…
What changed my mind as a Java/Kotlin developer?

Lesson 1: Asking the right questions

Lesson 2: There is no best language but tradeoffs

Lesson 3: Adapting a new language extends your views on other languages

Copyright © 2025 Xebia. All rights reserved